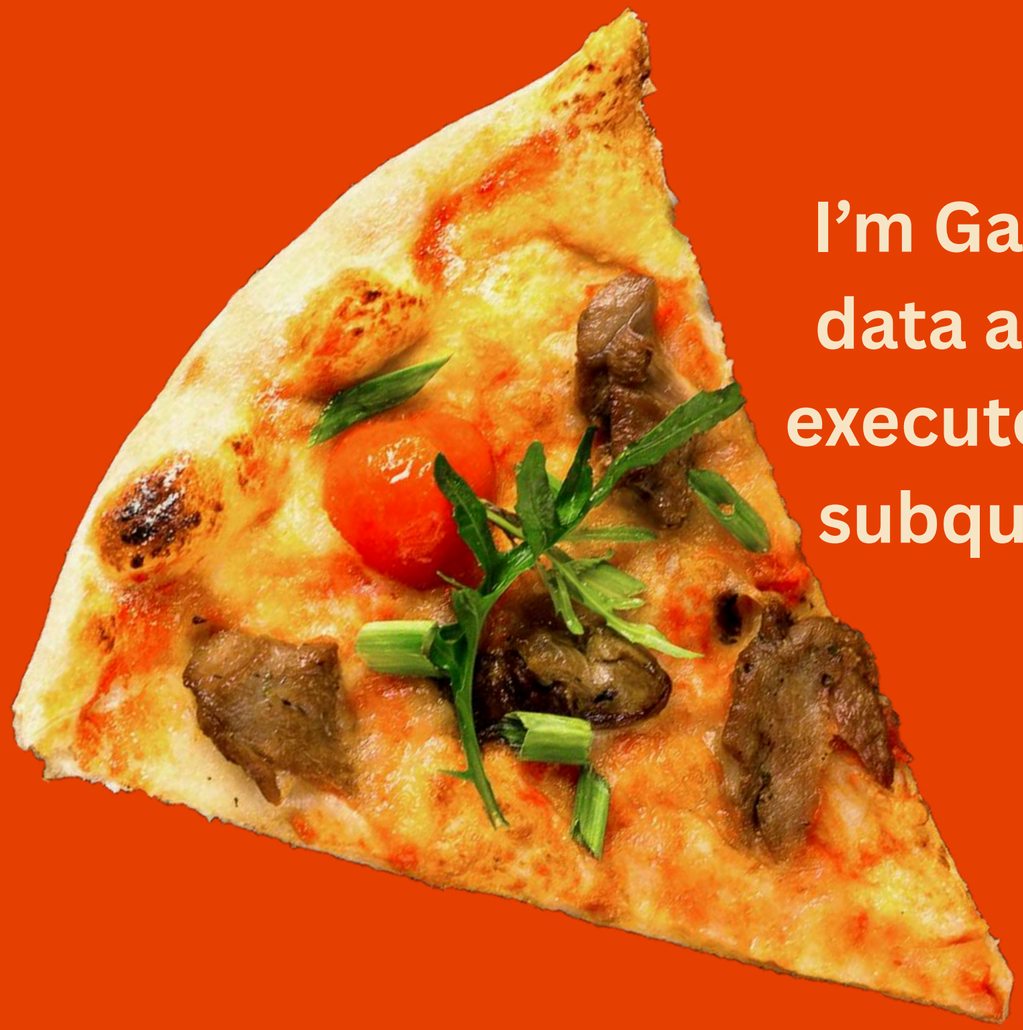
PIZZA SALES TRENDS SQL

A Delicious Journey

HELLO!



I’m Gayatri, In this project, I performed end-to-end data analysis using MySQL Workbench. I wrote and executed SQL queries involving JOINs, aggregations, subqueries, and window functions to analyze pizza sales data.

PROJECT OVERVIEW: PIZZA SALES TRENDS SQL

Key contributions:

- Analyzed sales trends, order patterns, and revenue distribution
- Identified top-selling pizzas, categories, and peak order hours
- Calculated key metrics like total revenue, average orders, and percentage contribution
- Derived business insights to support data-driven decision-making

Retrieve the total number of orders placed

```sql
-- Retrieve the total number of orders placed
SELECT COUNT(order_id) AS total_orders
FROM orders;
```

Calculate the total number of revenue generated from pizza sales

```sql
-- Calculate the total number of revenue generated from pizza sales
SELECT
    ROUND(SUM(order_details.quantity * pizzas.price),
            2) AS total_sales
FROM
    order_details
        JOIN
    pizzas ON order_details.pizza_id = pizzas.pizza_id;
```

IDENTIFY THE HIGHEST PRICED PIZZA

```
-- Identify the Highest Priced pizza
select pizza_types.name, pizzas.price
from pizza_types join pizzas
on pizza_types.pizza_type_id = pizzas.pizza_type_id
order by price desc
limit 1;
```

Identify the most common pizza size ordered

```sql
-- Identify the most common pizza size ordered
SELECT
    pizzas.size,
    COUNT(order_details.order_details_id) AS order_count
FROM
    pizzas
        JOIN
    order_details ON pizzas.pizza_id = order_details.pizza_id
GROUP BY pizzas.size
ORDER BY order_count DESC;
```

List the top 5 most ordered pizza types along with their quantities

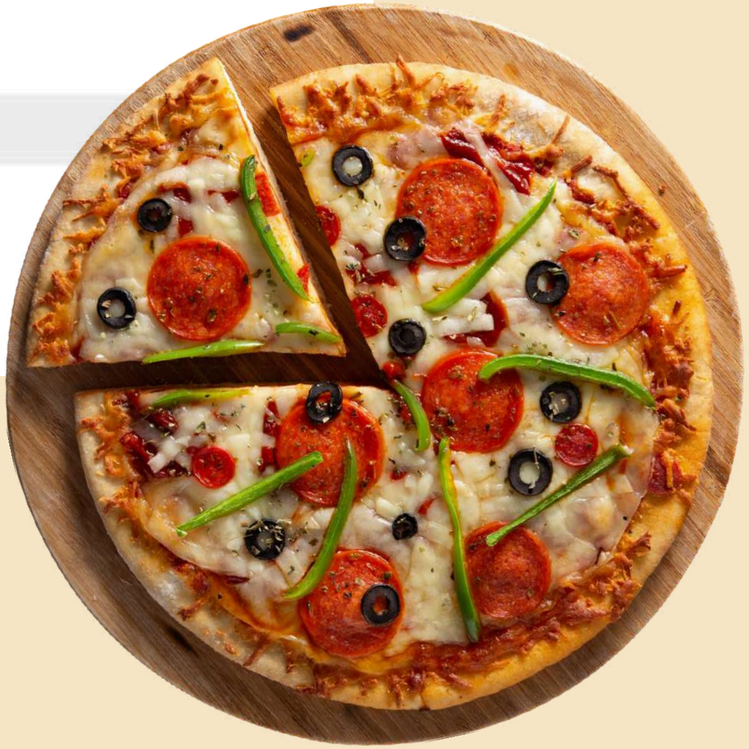
```sql
-- List the top 5 most ordered pizza types
-- along with their quantities

SELECT
    pizza_types.name, SUM(order_details.quantity) AS quantity
FROM
    pizza_types
        JOIN
    pizzas ON pizza_types.pizza_type_id = pizzas.pizza_type_id
        JOIN
    order_details ON order_details.pizza_id = pizzas.pizza_id
GROUP BY pizza_types.name
ORDER BY quantity DESC
LIMIT 5;
```

Join the necessary tables to find the total quantity of each pizza Category ordered

```sql
-- Join the necessary tables to find the total quantity of each pizza Category ordered
SELECT
    pizza_types.category,
    SUM(order_details.quantity) AS quantity
FROM
    pizza_types
        JOIN
    pizzas ON pizza_types.pizza_type_id = pizza_types.pizza_type_id
        JOIN
    order_details ON order_details.pizza_id = pizzas.pizza_id
GROUP BY pizza_types.category
ORDER BY quantity DESC;
```

Determine the distributions of orders by hour of the day

```sql
-- Determine the distributions of orders by hour of the day
SELECT
    HOUR(order_time) AS hours, COUNT(order_id) AS order_count
FROM
    orders
GROUP BY (order_time);
```



join relevant tables to find the catagorywise distribution of pizzas

```sql
-- join relevant tables to find the catagorywise distribution of pizzas
SELECT category, COUNT(name) FROM pizza_types
GROUP BY category;
```

Group the orders by date and calculate the average number of pizzas ordered per day

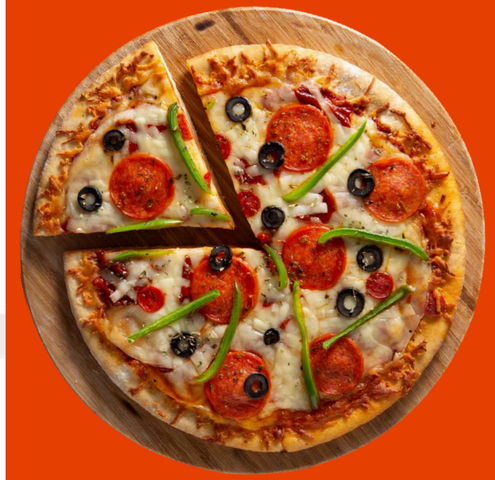
```sql
-- Group the orders by date and calculate the average number of pizzas ordered per day
SELECT
    ROUND(AVG(quantity), 0) AS avg_pizza_ordered_per_day
FROM
    (SELECT
        orders.order_date, SUM(order_details.quantity) AS quantity
    FROM
        orders
    JOIN order_details ON orders.order_id = order_details.order_id
    GROUP BY orders.order_date) AS order_quantity;
```

Determine the top 3 most ordered pizza types based on the revenue

```sql
-- Determine the top 3 most ordered pizza types based on the revenue
SELECT pizza_types.name,
SUM(order_details.quantity * pizzas.price) AS revenue
from pizza_types join pizzas
ON pizzas.pizza_type_id = pizza_types.pizza_type_id
JOIN order_details
ON order_details.pizza_id = pizzas.pizza_id
group by pizza_types.name order by revenue desc limit 3;
```

Calculate the percentage contribution of each pizza type to total revenue

```sql
-- Calculate the percentage contribution of each pizza type to total revenue
SELECT
    pizza_types.category,
    ROUND((SUM(order_details.quantity * pizzas.price) / (SELECT
                    ROUND(SUM(order_details.quantity * pizzas.price),
                                2) AS total_sales
                FROM
                    order_details
                        JOIN
                    pizzas ON pizzas.pizza_id = order_details.pizza_id)) * 100,
            2) AS revenue
FROM
    pizza_types
        JOIN
    pizzas ON pizzas.pizza_type_id = pizza_types.pizza_type_id
            JOIN
      order_details ON order_details.pizza_id = pizzas.pizza_id
 GROUP BY pizza_types.category
 ORDER BY revenue DESC;
```



Analyze the cumulative revenu generated over time

```
 -- Analyze the cumulative revenu generated over time
SELECT order_date,
SUM(revenue) OVER ( ORDER BY order_date) as cum_revenue
FROM
(SELECT orders.order_date,
SUM(order_details.quantity * pizzas.price) AS revenue
FROM order_details join pizzas
ON order_details.pizza_id = pizzas.pizza_id
JOIN orders
ON orders.order_id = order_details.order_id
group by orders.order_date) as sales;
```

Determine the top 3 most ordered pizza types based on revenue for each pizza category

```sql
-- Determine the top 3 most ordered pizza types based on revenue for each pizza category
SELECT name, revenue
FROM
(SELECT category, name, revenue,
rank() over(partition by category order by revenue desc ) as rn
FROM
(SELECT
    pizza_types.category,
    pizza_types.name,
    SUM((order_details.quantity) * pizzas.price) AS revenue
FROM
    pizza_types
        JOIN
    pizzas ON pizza_types.pizza_type_id = pizzas.pizza_type_id
        JOIN
    order_details ON order_details.pizza_id = pizzas.pizza_id
GROUP BY pizza_types.category , pizza_types.name) AS a) AS b
WHERE rn <= 3;
```

THANK YOU